

Optimal Cargo Management for Flights



Understanding the Problem Statement

01 .

Input:

List of ULDs with dimensions and weight limits, list of package with dimensions, weight, type (priority/economy) and penalties

02 .

Constraints:

Packages must fit orthogonally into ULDs without intersection or protrusion and within weight limits.

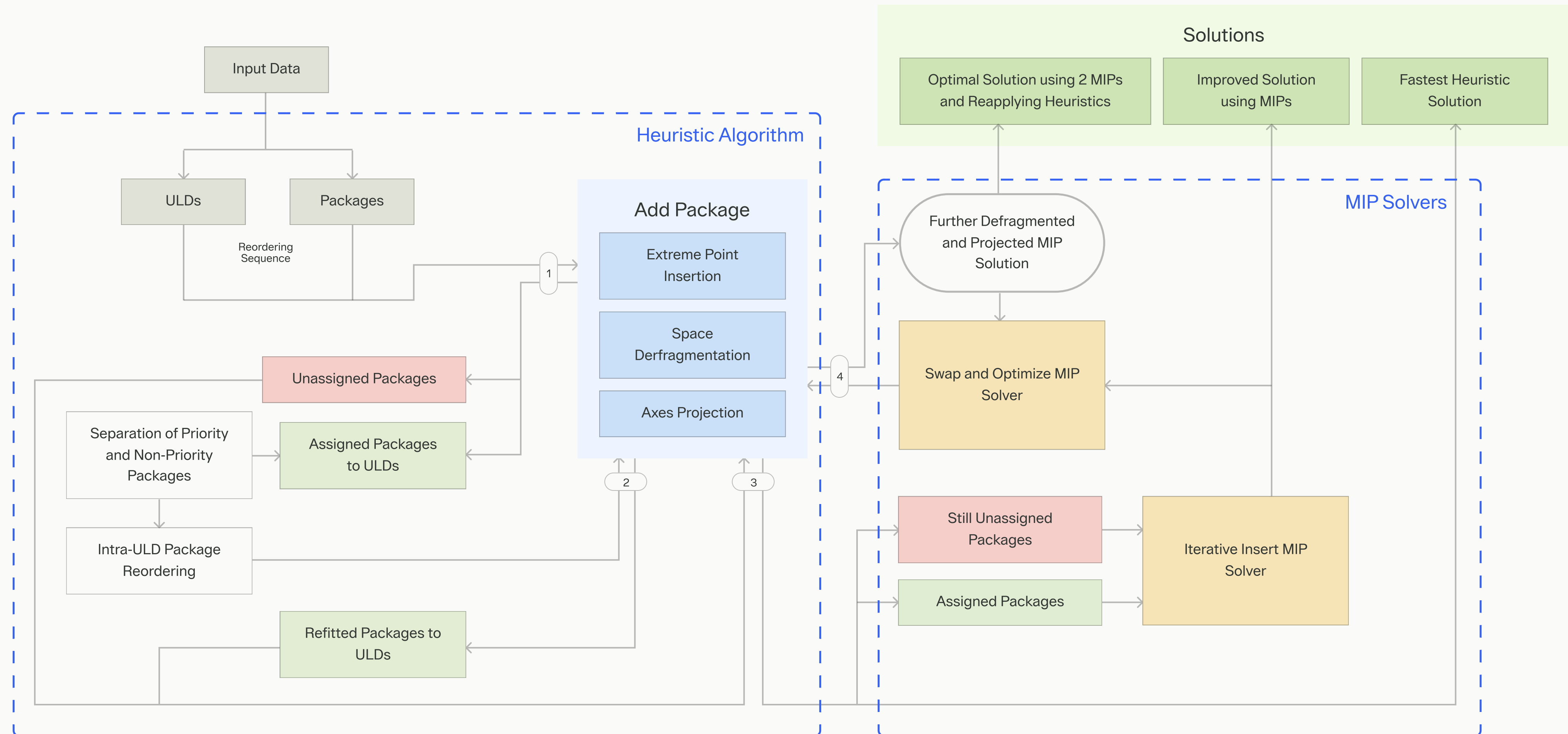
Priority packages must be packed.

03 .

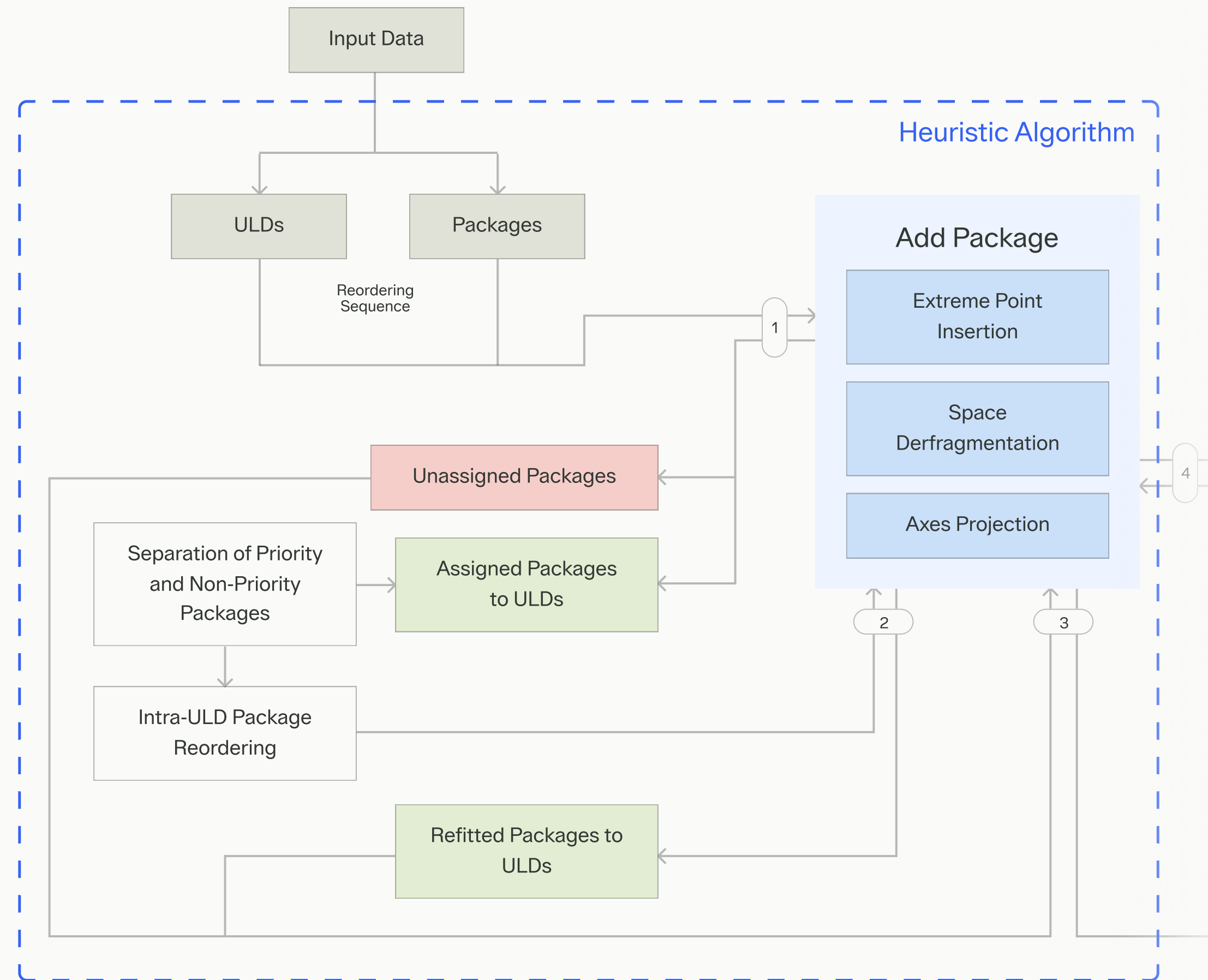
Goal:

Minimize the cost of unshipped economy packages while minimizing the spread of priority packages across ULDs (penalty constant K)

Our Pipeline: A Hybrid Solution



Heuristics



Extreme Points

01 .

Place first package at the origin (front-left-bottom corner)

03 .

Select next extreme point based on minimum Euclidean distance to origin

02 .

Generate new extreme points from the bounding box corners

04 .

More efficient than using just container corners, reducing wasted space



Axes Projection

01 .

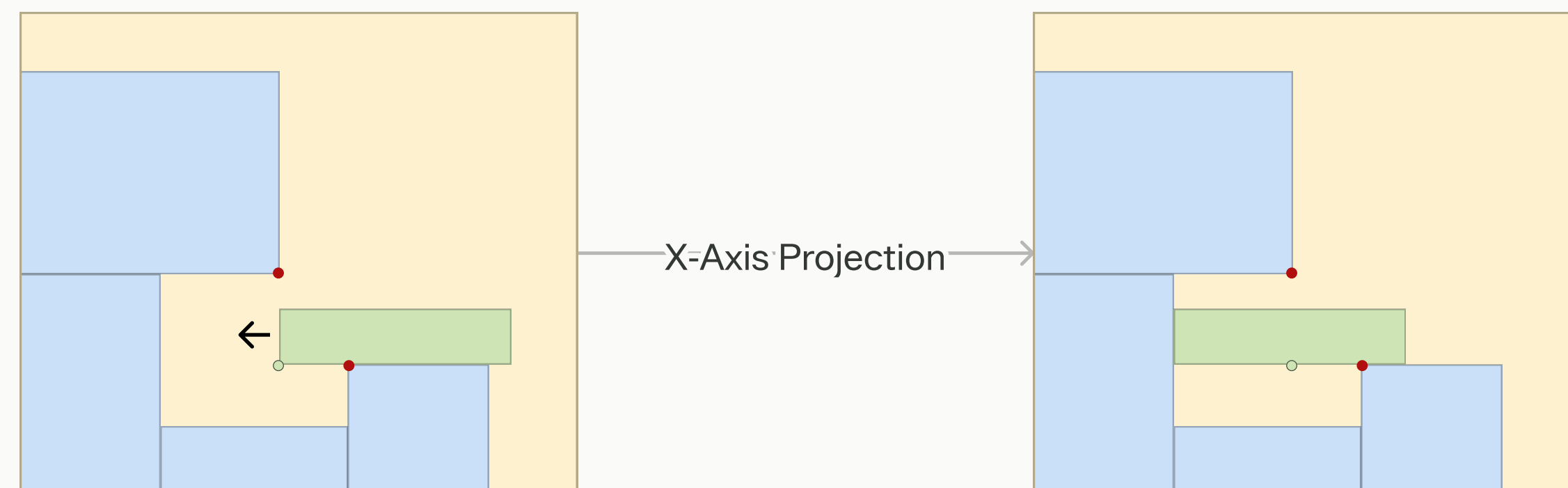
After placing a package, identify unused free space behind it

02 .

Push the package back toward the origin to eliminate gaps

03 .

Maximizes space utilization by reducing unnecessary voids in the layout



Space Defragmentation

01 .

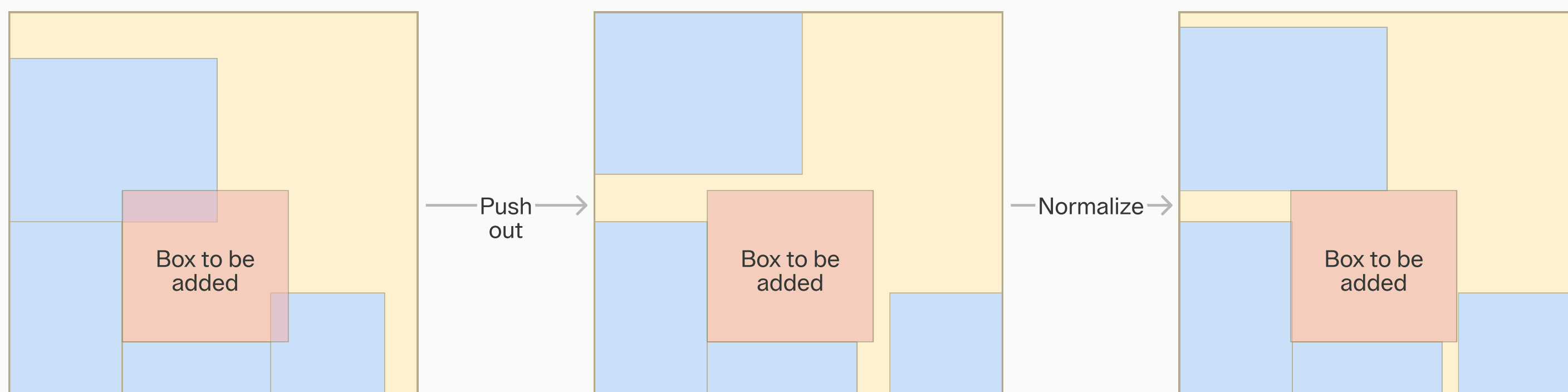
Calculate per package moving limit

02 .

Inflate gaps to make contiguous space and swap smaller packages with larger ones to fill voids.

03 .

Normalize all packages into their optimal positions for tighter packing.



Sequence Order

01 .

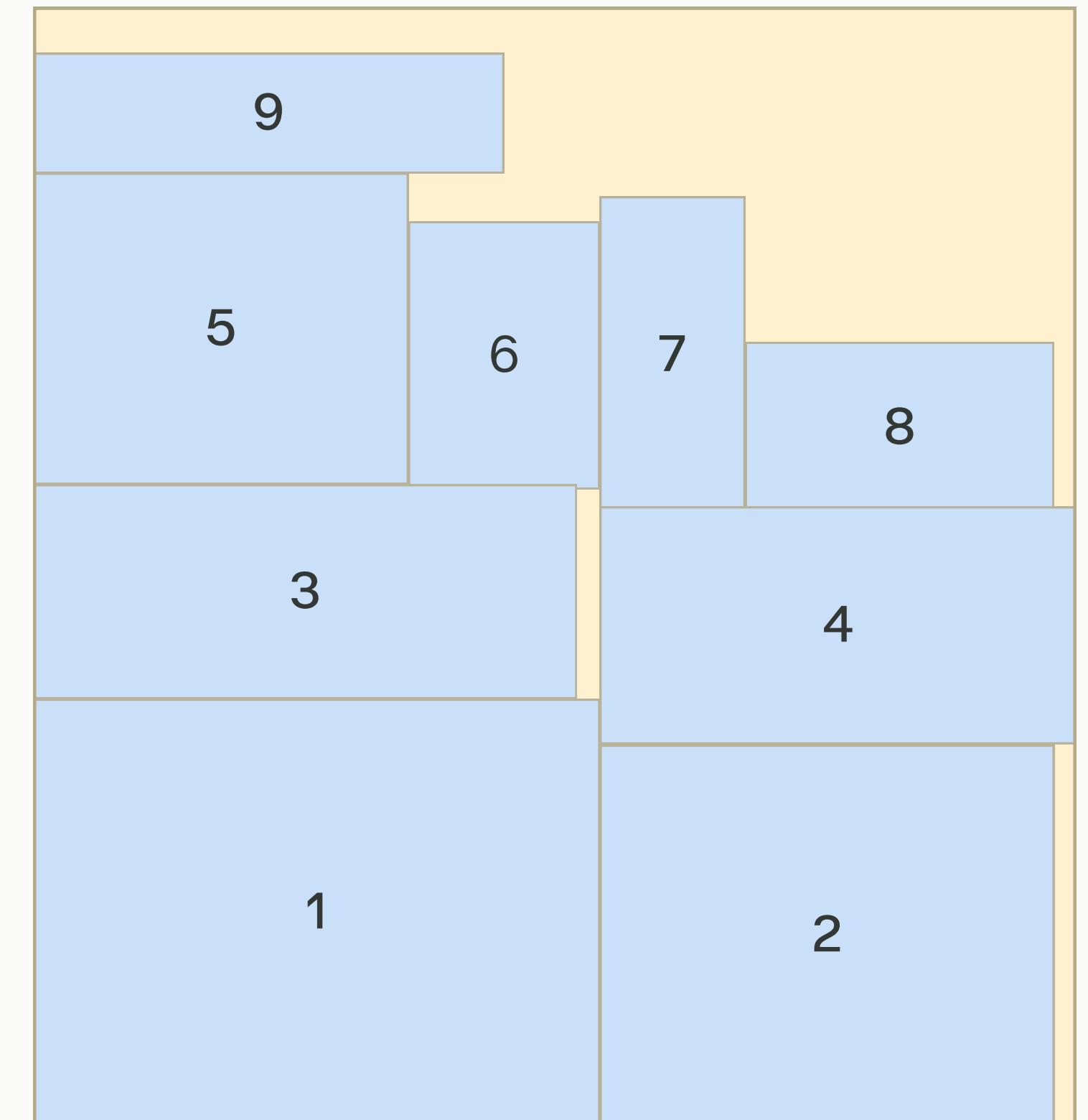
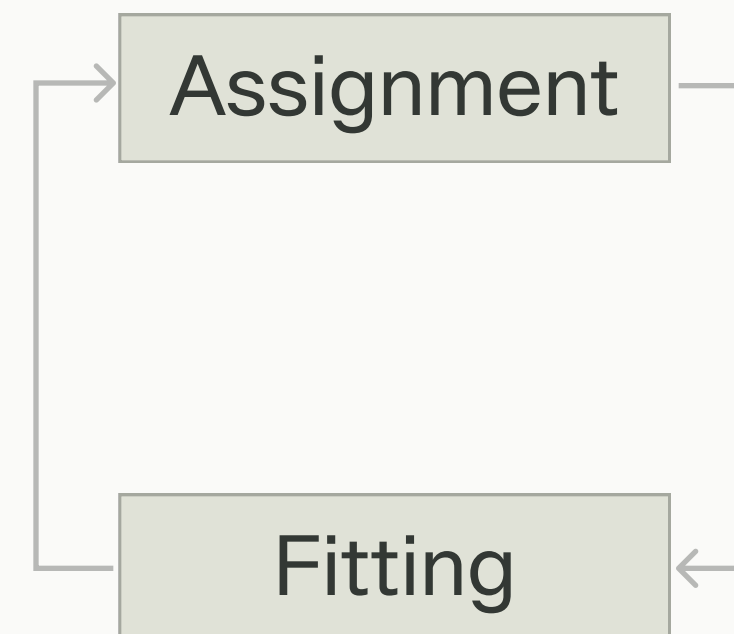
Packing efficiency depends a lot on the initial order in which packages are assigned.

03 .

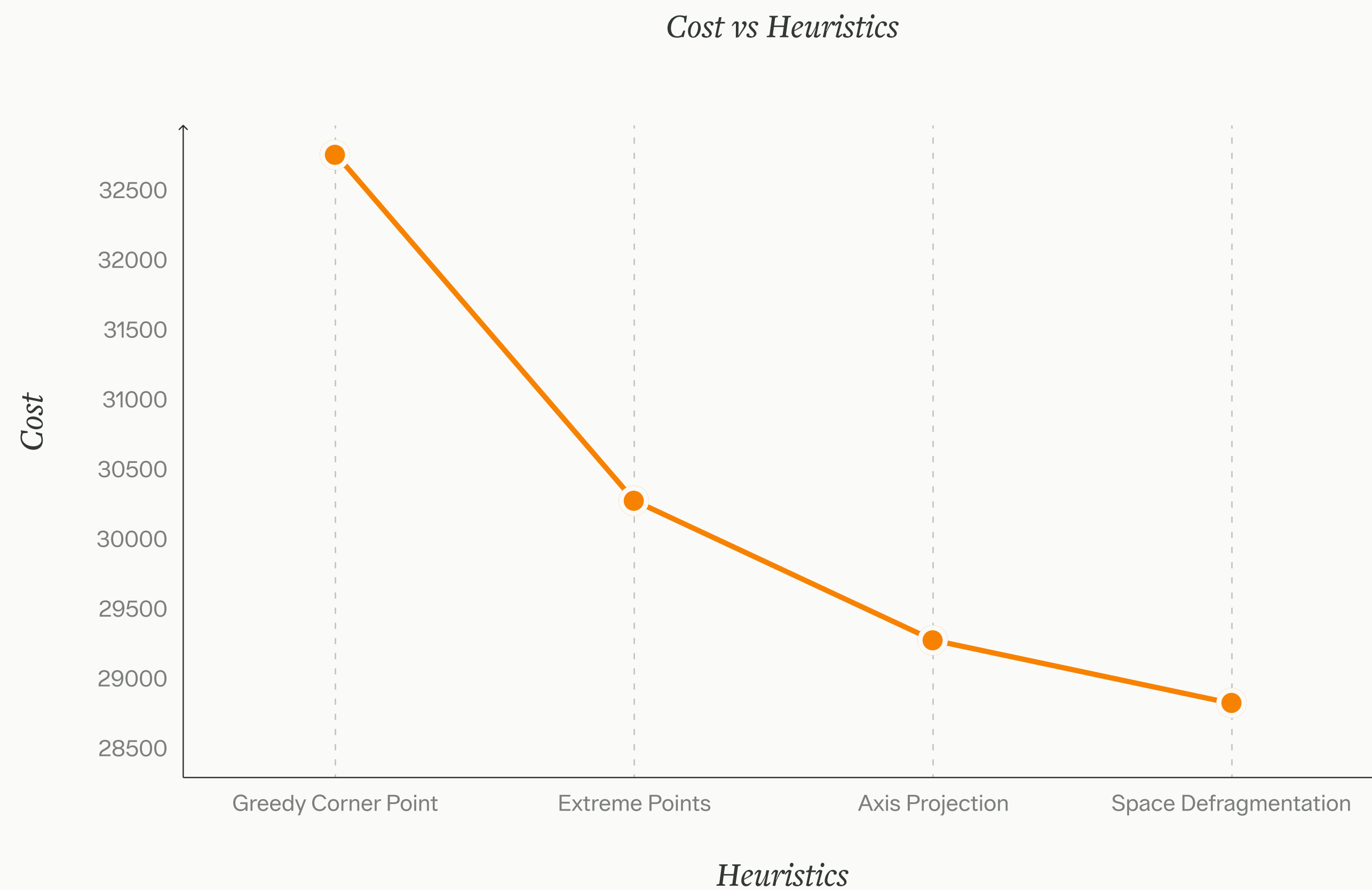
Arrange them based on a height-area heuristic while fitting to achieve tighter packing.

02 .

For assignment, we prioritize high-cost, low-volume packages to pack valuable items first.

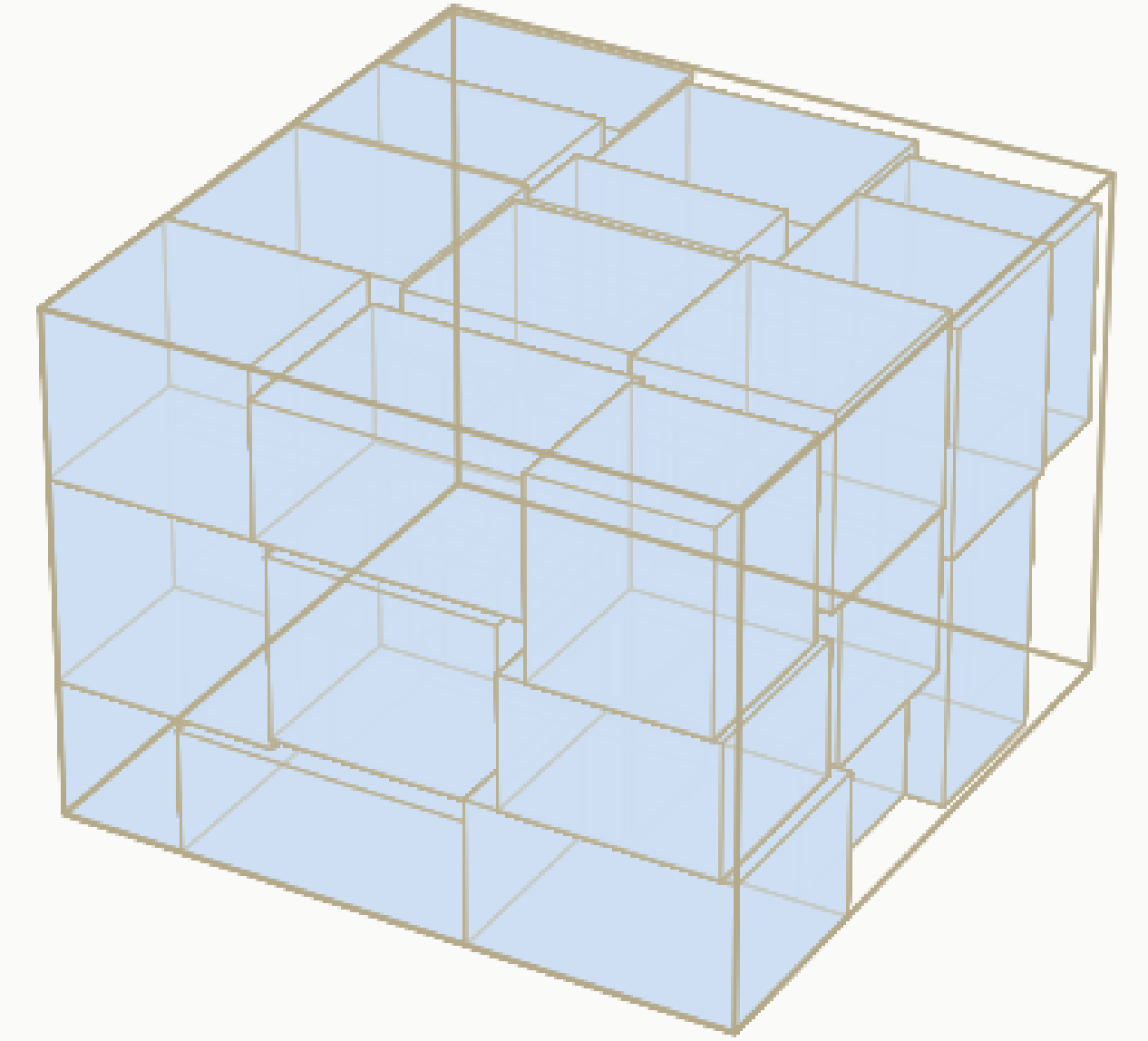


Progressive cost improvement

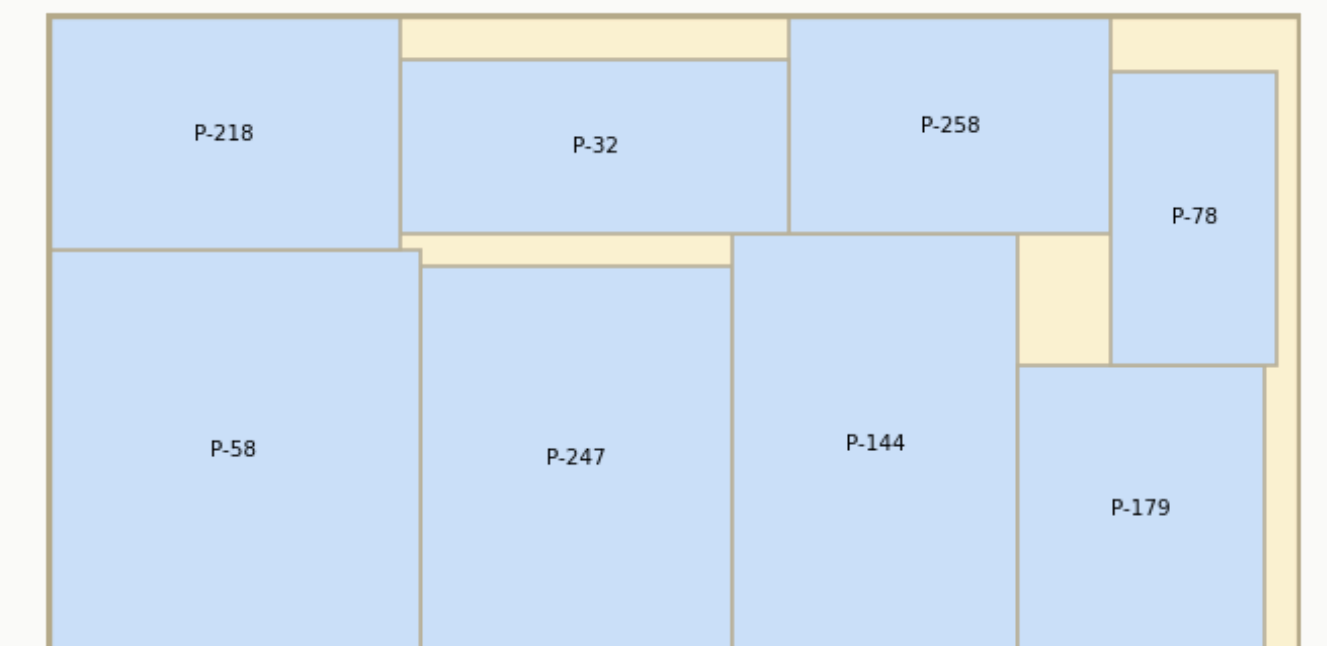


ULD Plots after Heuristics

- Fragmented spaces still present
- Heuristics cannot fully resolve random fragmentation
- Need some other method to address this issue



ULD Packing



Cross Section along the length, containing fragments

MIP based Optimizers

What is MIP?

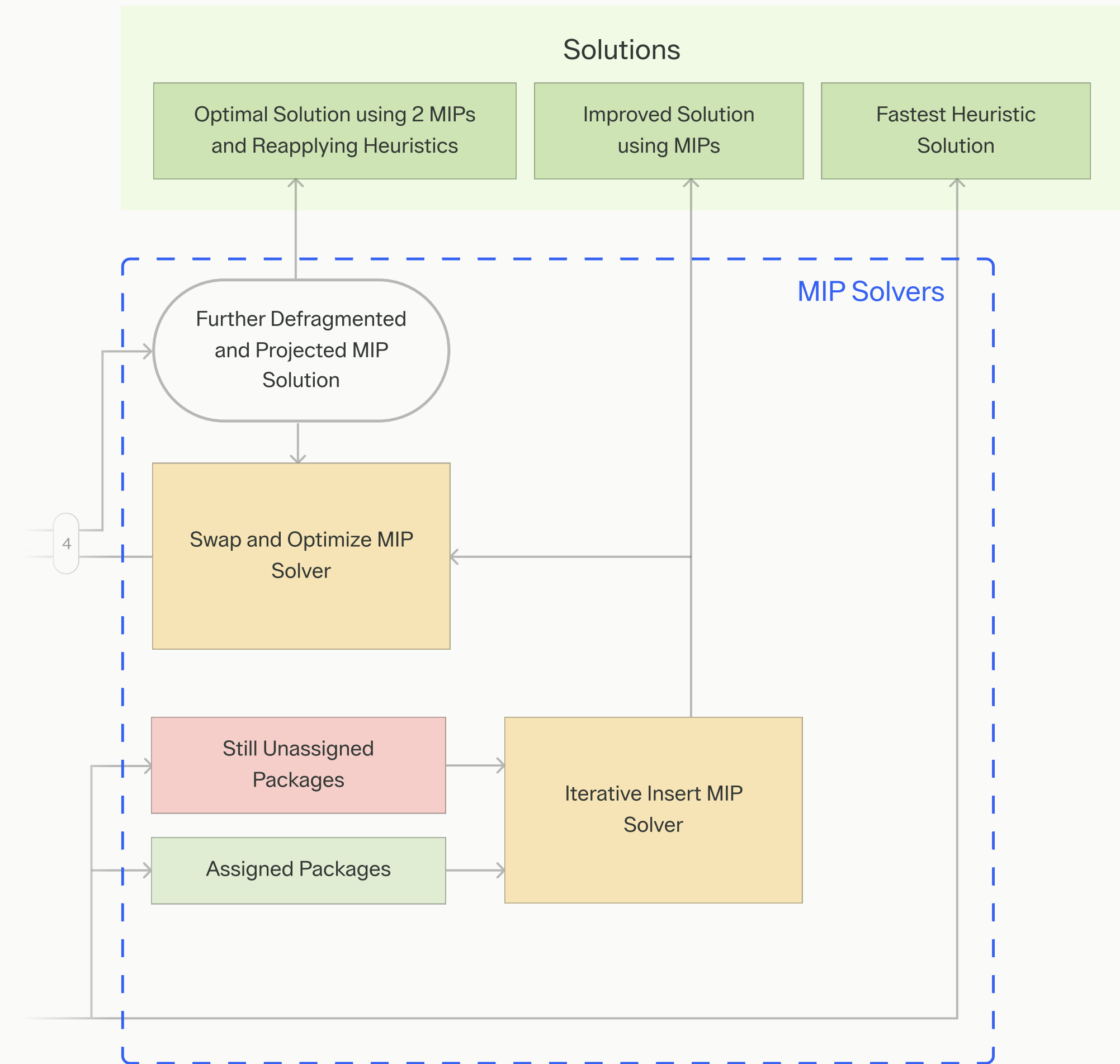
Mixed Integer Programming (MIP)

A mathematical model with constraints and an objective function to optimize (minimize/maximize).

Solved using Branch and Bound Algorithm:

Builds a search tree, creating branches for each integer variable.

NP-Hard: Impractical runtime for large inputs.



Constraints

01 .

No two packages should overlap:

$$x_i + p_i \cdot l_{xi} + q_i \cdot w_{xi} + r_i \cdot h_{xi} \leq x_k + (1 - a_{ik}) \cdot M$$

$$x_k + p_k \cdot l_{xk} + q_k \cdot w_{xk} + r_k \cdot h_{xk} \leq x_i + (1 - b_{ik}) \cdot M$$

$$y_i + q_i \cdot w_{yi} + p_i \cdot l_{yi} + r_i \cdot h_{yi} \leq y_k + (1 - c_{ik}) \cdot M$$

$$y_k + q_k \cdot w_{yk} + p_k \cdot l_{yk} + r_k \cdot h_{yk} \leq y_i + (1 - d_{ik}) \cdot M$$

$$z_i + r_i \cdot h_{zi} + q_i \cdot w_{zi} + p_i \cdot l_{zi} \leq z_k + (1 - e_{ik}) \cdot M$$

$$z_k + r_k \cdot h_{zk} + q_k \cdot w_{zk} + p_k \cdot l_{zk} \leq z_i + (1 - f_{ik}) \cdot M$$

*For all pairs of
packages (i, k)*

Constraints

02 .

Package does not protrude out of ULD:

$$a_{ik} + b_{ik} + c_{ik} + d_{ik} + e_{ik} + f_{ik} \geq s_{ij} + s_{kj} - 1$$

$$x_i + p_i \cdot l_{xi} + q_i \cdot w_{xi} + r_i \cdot h_{xi} \leq L_j + (1 - s_{ij}) \cdot M$$

$$y_i + q_i \cdot w_{yi} + p_i \cdot l_{yi} + r_i \cdot h_{yi} \leq W_j + (1 - s_{ij}) \cdot M$$

$$z_i + r_i \cdot h_{zi} + q_i \cdot w_{zi} + p_i \cdot l_{zi} \leq H_j + (1 - s_{ij}) \cdot M$$

*For all pairs of
packages (i, k) and
ULDs j*

03 .

Weight Constraint:
For all packages i and ULDs j

$$\sum_{i=1}^n s_{ij} \cdot W_i \leq W_j$$

Constraints

03 .

Relative Position Constraints:

For all pairs of packages (i, k) and ULDs j

$$a_{ik} + b_{ik} + c_{ik} + d_{ik} + e_{ik} + f_{ik} \geq s_{ij} + s_{kj} - 1$$

04 .

Supporting Constraints:

$$\sum_{i=1}^N s_{ij} \leq M \cdot n_j$$

*For all packages i
and ULDs j*

05 .

Package Assignment Guarantee:

For all packages i and ULDs j

!!!modify right constraint to less than equal. divide 3 into two parts, one for each mip

$$\sum_{j=1}^m s_{ij} = 1$$

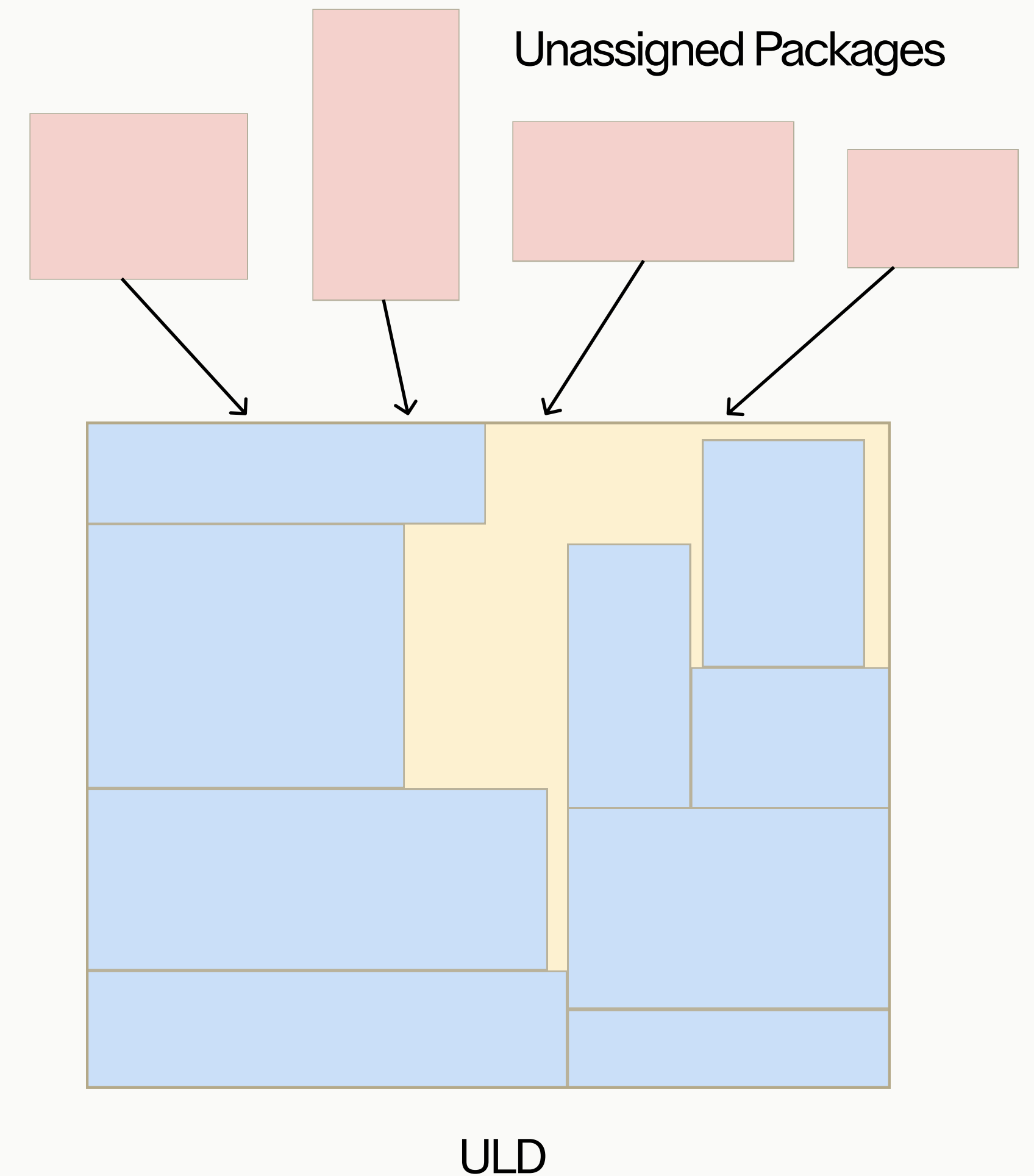
$$\sum_{j=1}^m s_{ij} \leq 1$$

Iterative Insert MIP

Tries to fit unassigned packages into ULDs.
No objective function, only feasibility checked

Workflow:

1. Select top few unassigned packages based on a priority metric:
 - Group by volume bins.
 - Sort within each bin by decreasing cost.
2. Iterate over all ULDs (sorted by free volume).
3. If insertion of unassigned package is feasible:
 - Refit the entire arrangement of the ULD including the unassigned package, which guarantee a better cost.



Swap and Optimize MIP

Naive objective function:

Minimize:

$$\left(\sum_{j=1}^m K \cdot \prod_{i=1}^n (x_{ij} \cdot p_i) + L \cdot \sum_{i=1}^n \left(1 - \sum_{j=1}^n x_{ij} \right) \right)$$

Reduced objective function:

Minimize:

$$\left(L \cdot \sum_{i=1}^n \left(1 - \sum_{j=1}^n x_{ij} \right) \right)$$

Swap and Optimize MIP

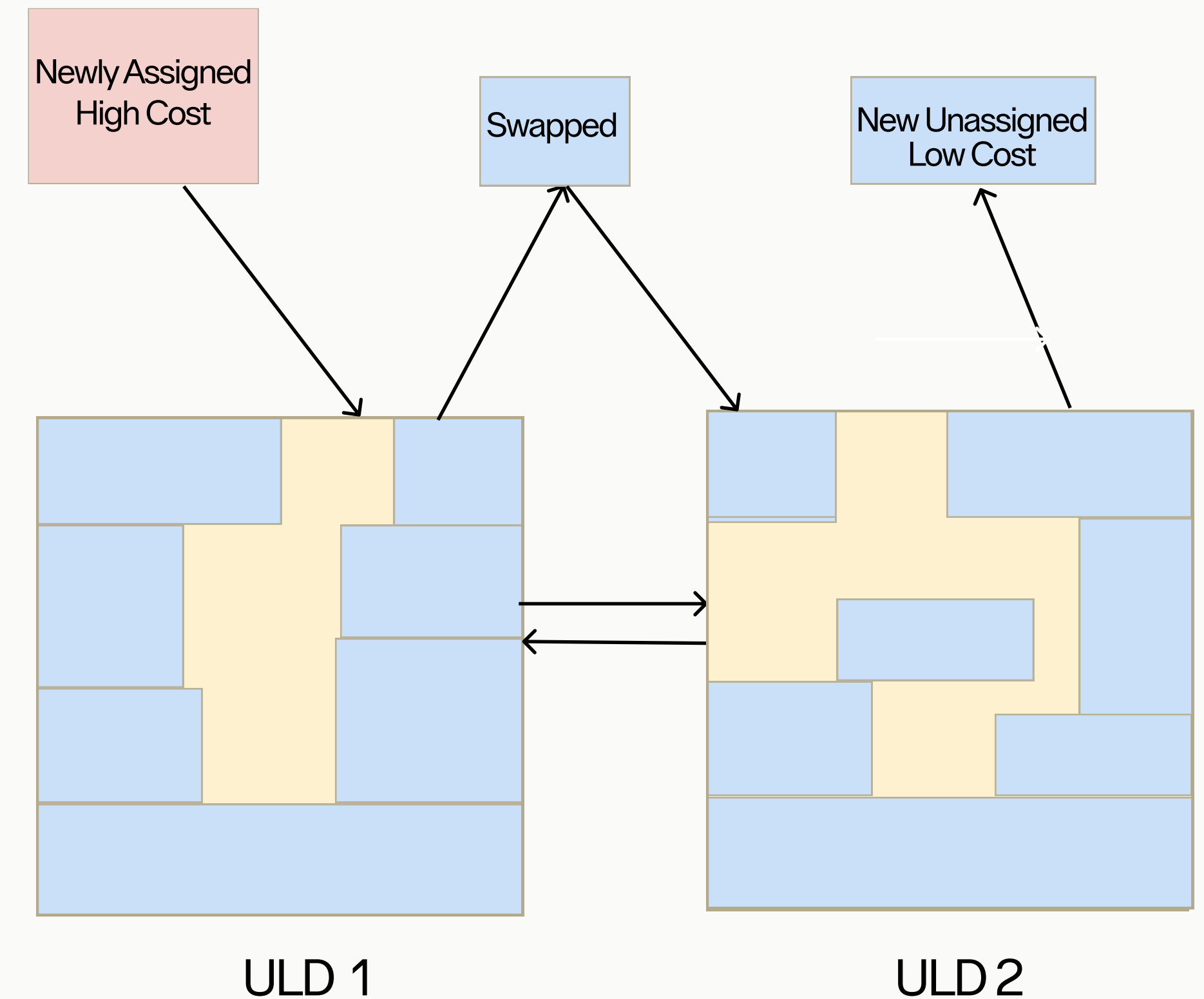
Generalized version of previous MIP over multiple ULDs

Objective function for cost introduced

Given much more freedom by also allowing inter-ULD swapping

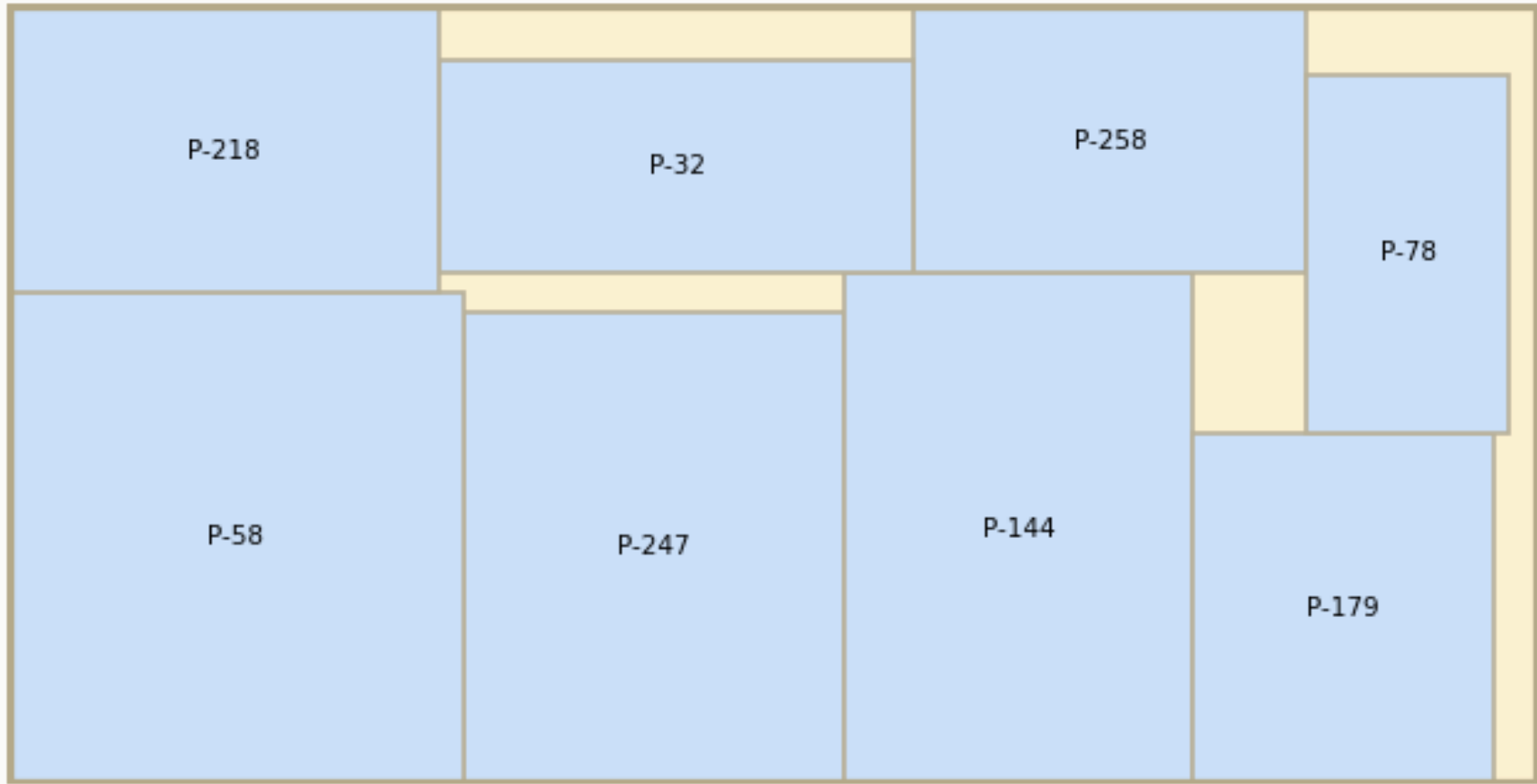
Workflow:

1. Select top few unassigned packages based on a priority metric
2. Select subset of ULDs and all their assigned packages
3. Now try to refit all these packages into the considered ULDs while minimizing the cost

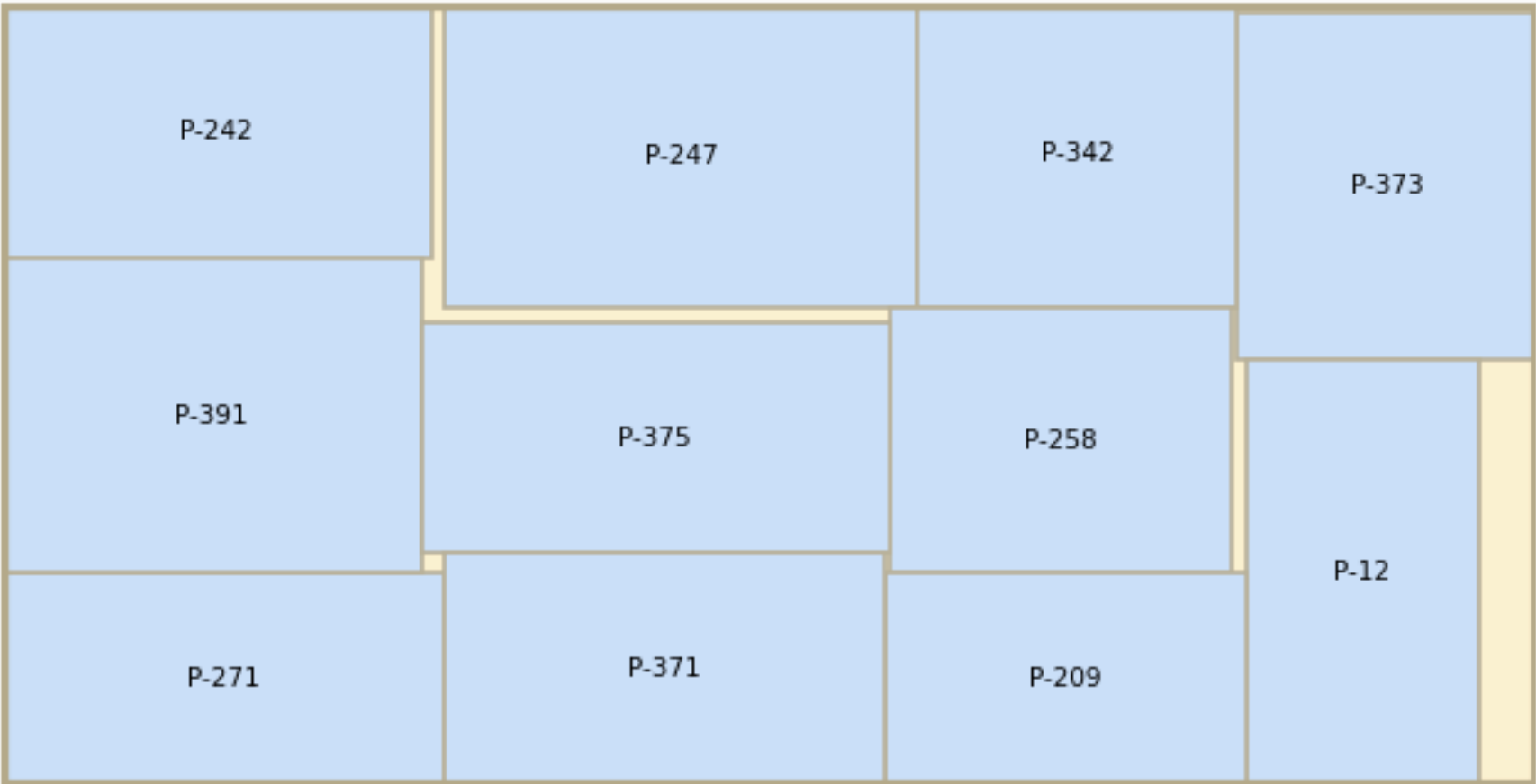


ULD Cross-Section Comparison

Cross section of ULD 1 along length



Only Heuristics



Both Heuristics and MIPs

Significant decrease in fragmented spaces after MIP

Completing the Pipeline

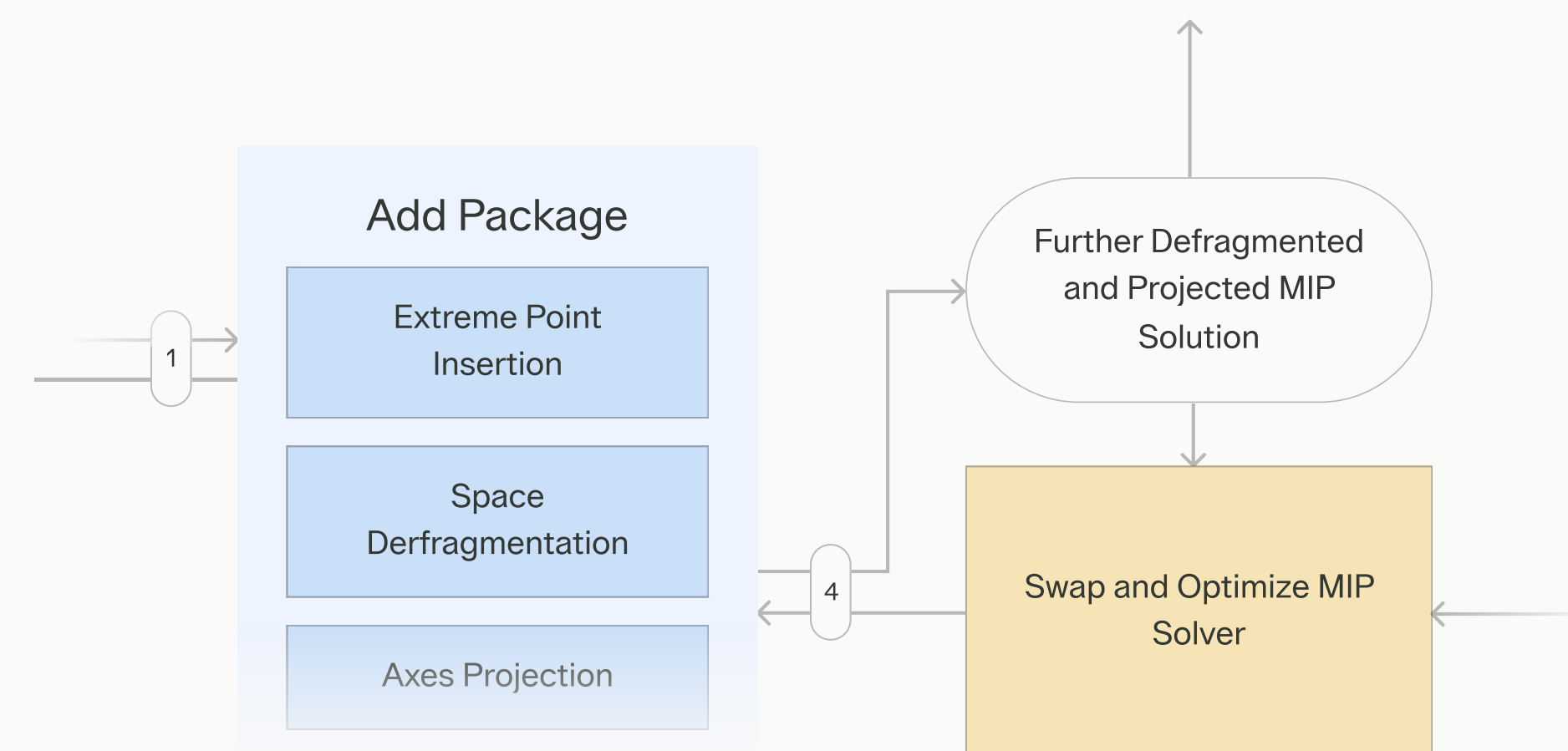
Final Adjustments:

Post-MIP heuristics applied:

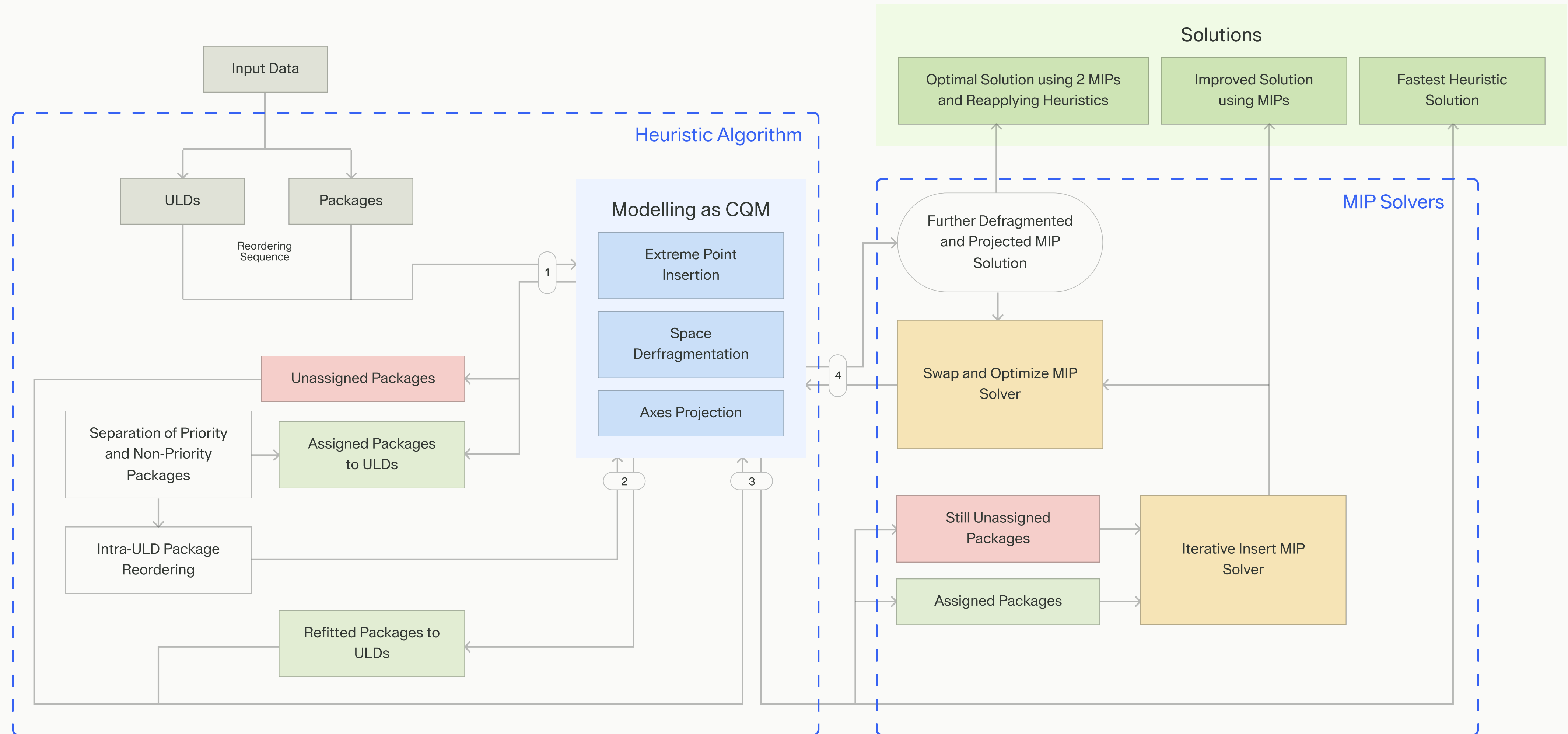
- Space defragmentation: Rearranges and reduces fragmented space.
- Axes projection: Ensures real-world constraints like package support and gravity.

According to the time available, we can reiterate over the pipeline multiple times.

Available time is divided among the various components of our pipeline efficiently.



Putting it all together



Results

Total number of packages assigned:
255

Number of ULDs containing priority packages:
3

Final Cost of our Solution:
27940

ULD ID	# Priority Packages	# Economy Package	% Free Volume	% Free Weight
ULD1	0	31	11.2%	25.2%
ULD2	0	27	10.8%	17.9%
ULD3	0	50	16.0%	0.5%
ULD4	33	12	14.8%	2.4%
ULD5	39	24	20.7%	11.3%
ULD6	31	8	12.0%	0.5%
Total	103	152	14.8%	9.0%

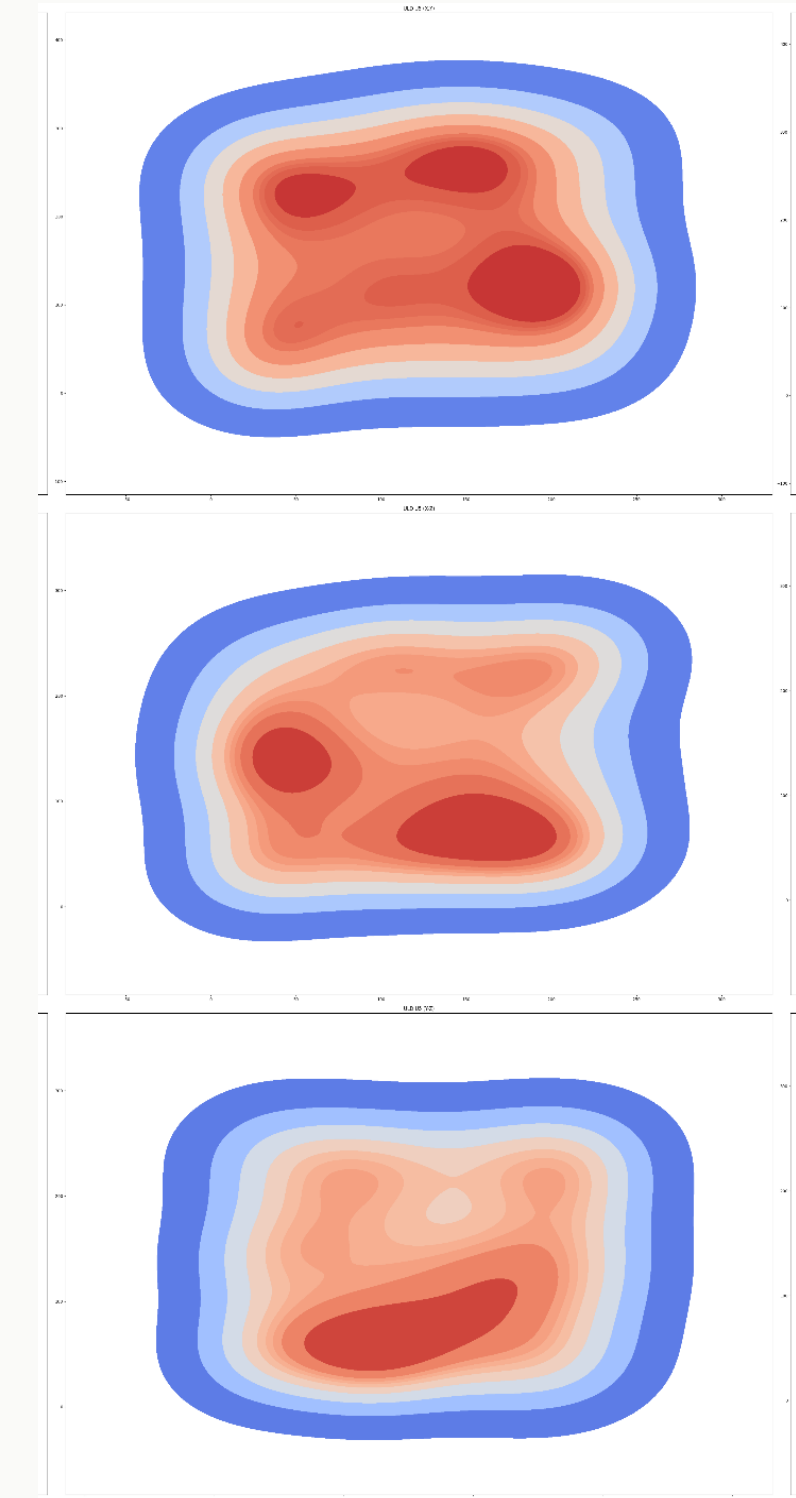
ULD Metrics

Advanced Analysis - Stability

We consider a package stable if more than 50% of its base area is supported by another package or the ground.

Over 85% of the packages in the final solution meet the stability criterion, and this certainly is an underestimation.

We can ensure 100% stability as well, at the expense of the overall cost.



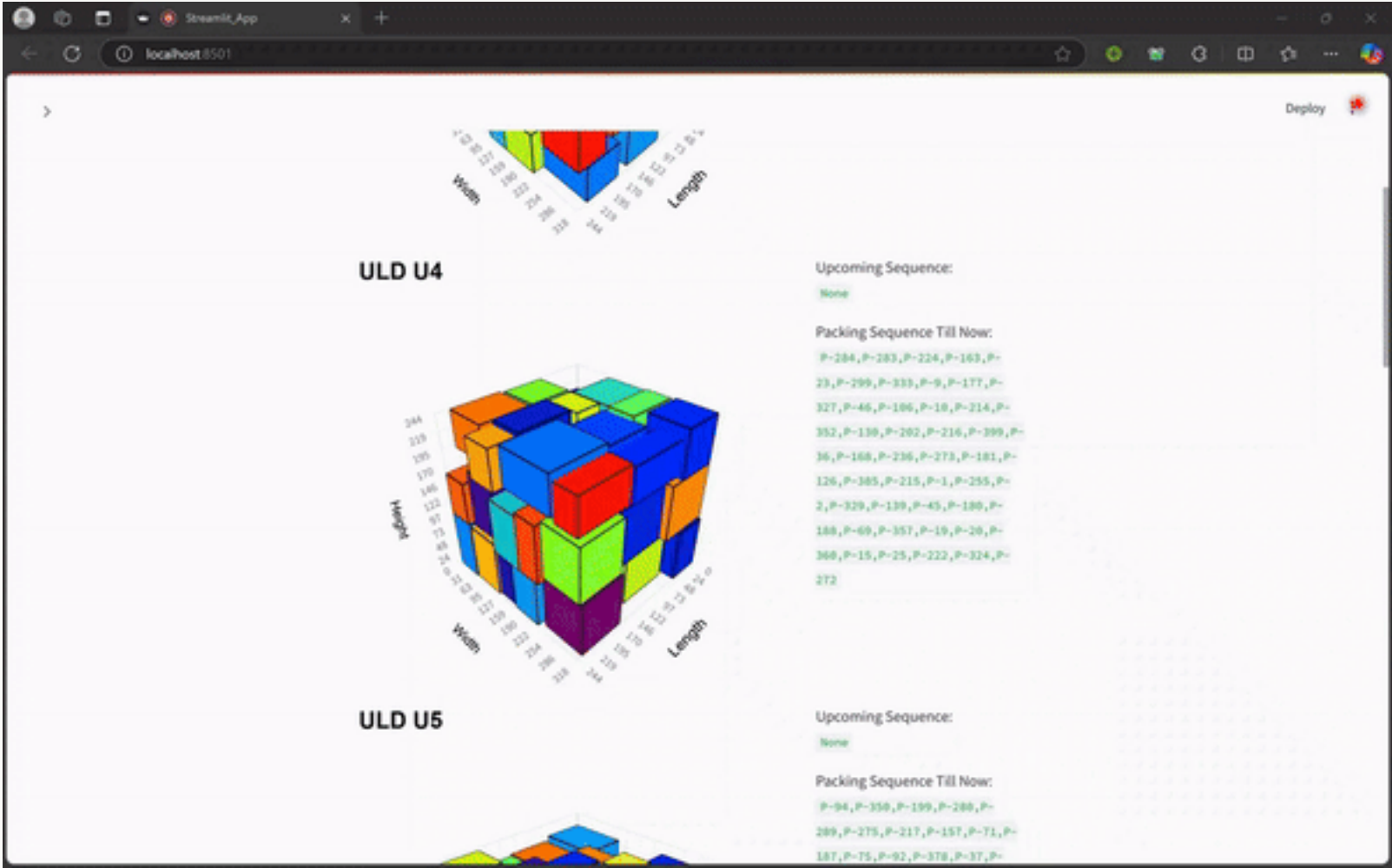
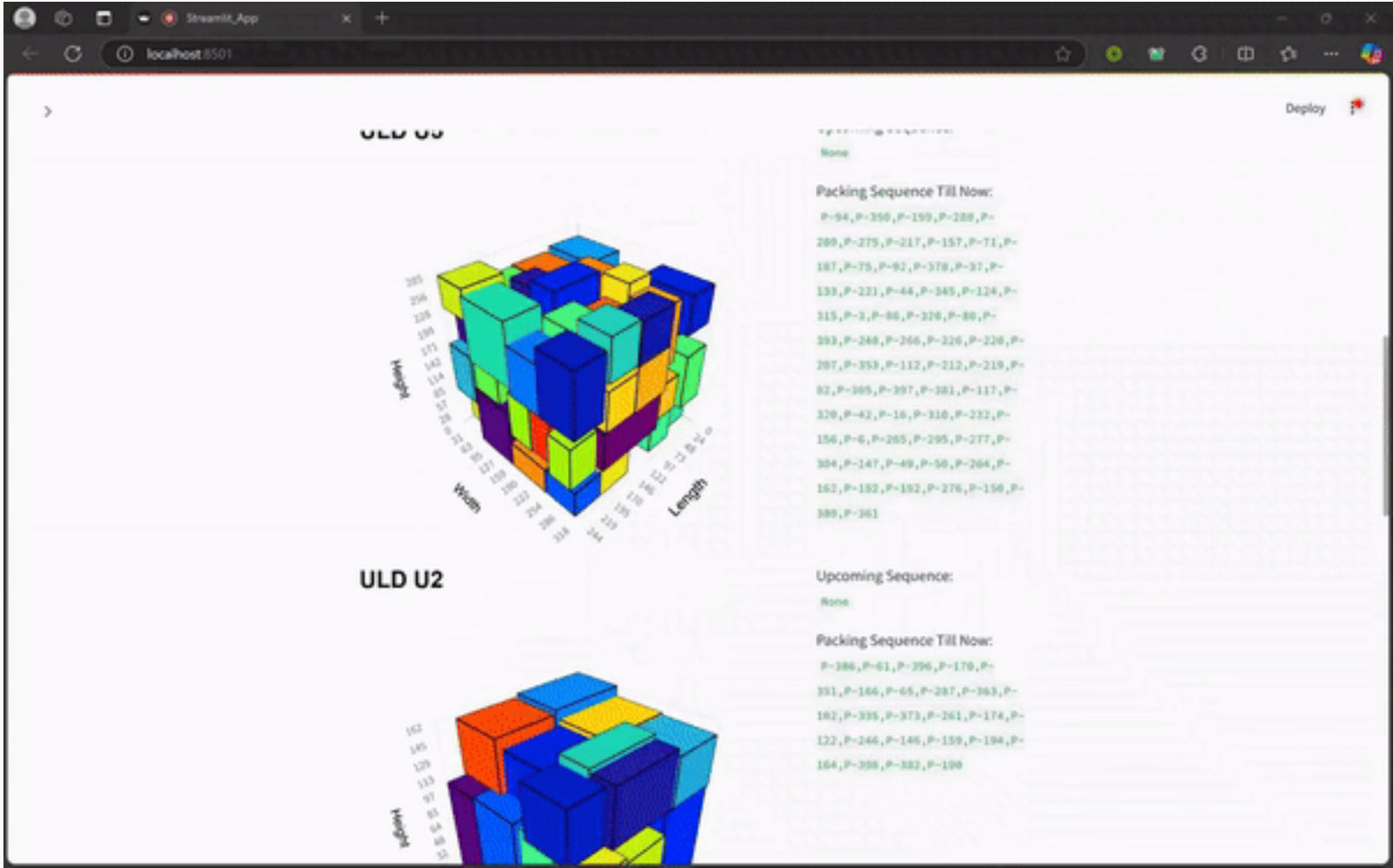
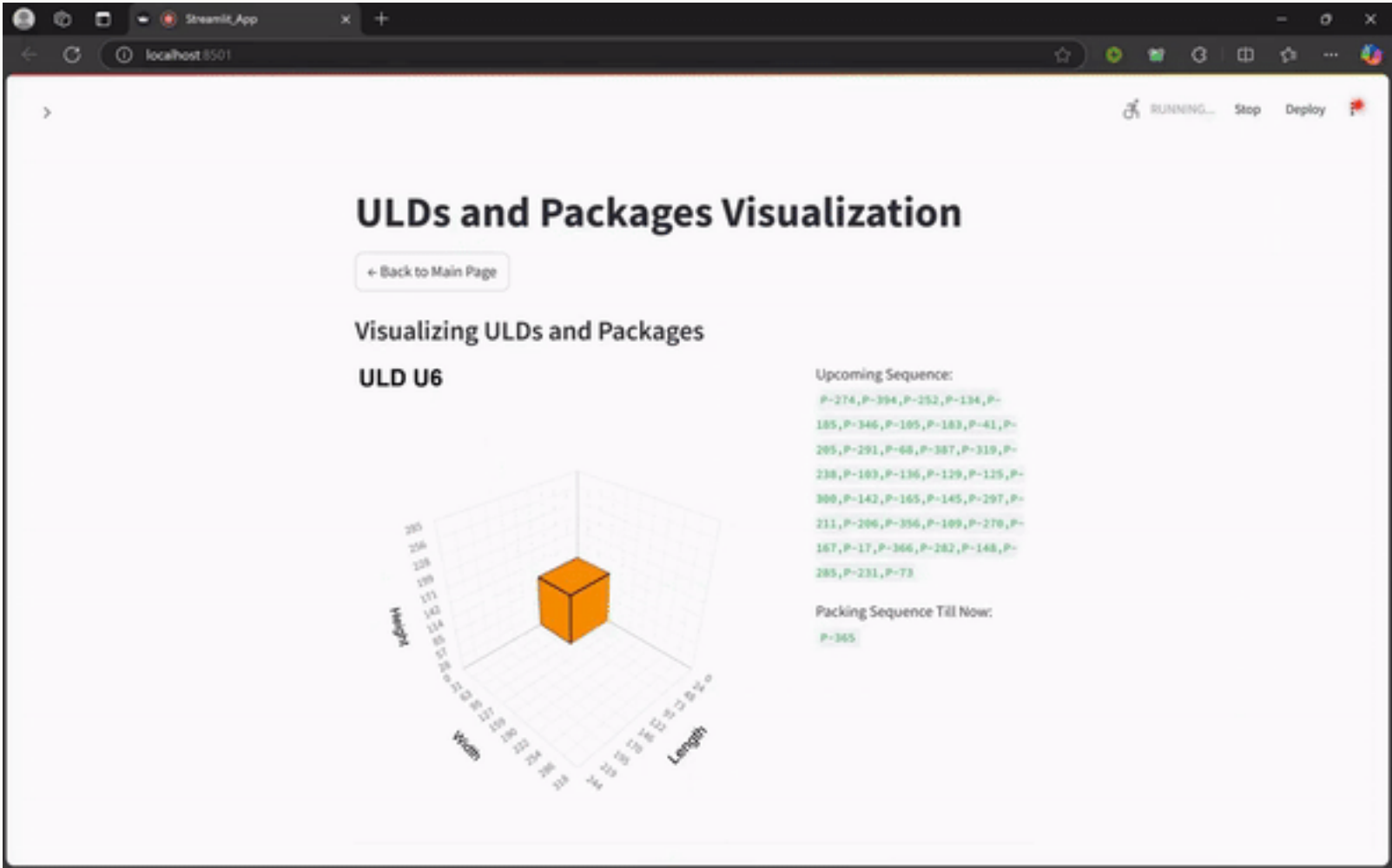
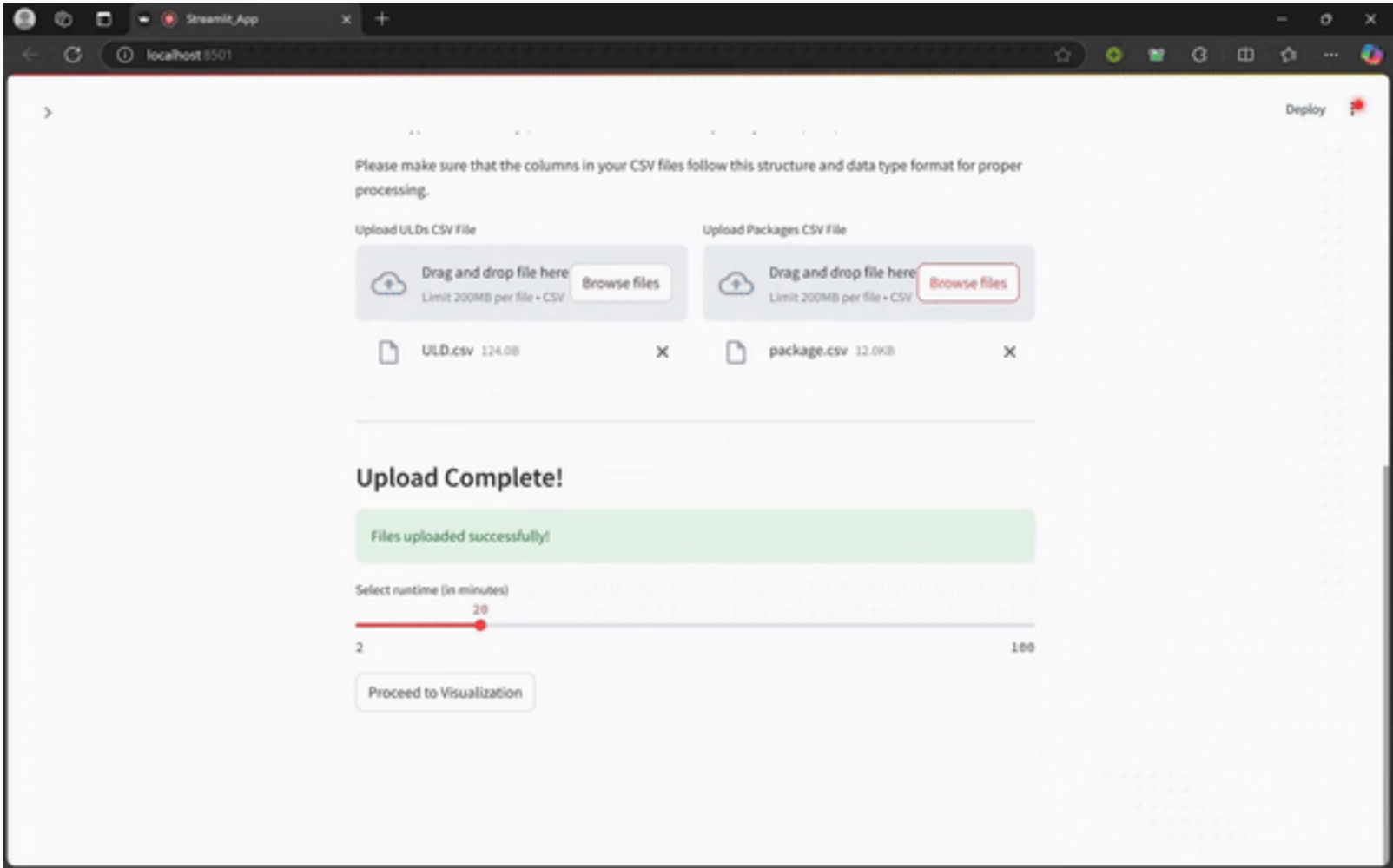
XY Plane

XZ Plane

YZ Plane

Centre of mass concentrated towards base
Weight evenly distributed on base

Web Application





Thank you.